

GAME PROGRAMMING COURSE — REFACTORING

リファクタリング入門

実際のゲームをその場で直しながら学ぶ、読みやすいコードの作り方

対象：1年生

RE /
FACTOR

PART 1

リファクタリングとは

何をする作業なのか、なぜ必要なのかを知る

1

リファクタリングとは

ゲームの動きは変えずに、コードの中身だけを整えること

変えないもの

ゲームの動き

操作感・見た目・ルール。遊ぶ人から見て、直す前と後で何も変わらない。

変えるもの

コードの書き方

変数や関数の名前、関数の分け方、同じ処理の重複、スクリプトの役割分担。

ゴール

あとで楽になる

読みやすい、直しやすい、バグの場所を見つけやすいコードにする。

POINT

新しい機能を足す、バグを直す、は別の作業。リファクタリングとは混ぜずに分けて行う。

理由

なぜ「動いているコード」を直すのか

「動く」と「あとから直せる」は別の話。困るのは、書いた後にもう一度そのコードを触るとき。



機能を足したいとき

どこに書けばいいかわからない。1か所足したら、別の場所が壊れる。



バグが出たとき

原因の場所を探すだけで時間がなくなる。直したつもりが、コピペした別の場所に同じバグが残る。



チームで作るとき

自分以外の人を読めない。そして1か月後の自分も、だいたい他人と同じくらい読めない。

POINT

コードは、書いている時間より読んでいる時間のほうが長い。だから「読みやすさ」に手間をかける価値がある。

この授業でやること

1年生が実際に作ったゲームのコードを、**その場で一緒にリファクタリング**していきます。

1

動かして確認

今の動きを覚えておく



2

気になる所を探す

読みにくい場所を見つける



3

1か所だけ直す

一度にまとめて直さない



4

もう一度動かす

同じ動きなら2に戻る

見てほしいところ

「どこが気になるか」→「なぜ直すのか」→「どう直すか」の順番。直し方より、気づき方を持ち帰る。

大事な前提

題材は「最後まで動くものを作りきった」作品。ダメ出しではなく、次の一歩を探す時間。

リファクタリングの鉄則

小さく直して、 毎回動かす。

一度にたくさん直すと、壊れたときにどこが原因か分からなくなる。

直す前に保存する

動いている状態のプロジェクトを複製しておく

1回の変更は1つだけ

名前を変える、関数に分ける、を同時にやらない

変だと思ったら戻す

動きが変わったら、粘らずに直前の状態へ戻る

PART 2

よくある「気になるコード」

自分のコードを見て「ここ、直せそう」と気づけるようになる

2

名前から意味が分からない

C# Player.cs

BEFORE

```
1 int a = 100;
2 float t;
3 bool flg;
4
5 void Func1(int x)
6 {
7     a -= x;
8 }
```

C# Player.cs

AFTER

```
1 int hp = 100;
2 float attackTimer;
3 bool isInvincible;
4
5 void TakeDamage(int damage)
6 {
7     hp -= damage;
8 }
```

- 変数は「何が入っているか」、関数は「何をするか」が名前だけで伝わるようにする
- bool は **is** や **has** で始めると、true のときの意味が分かりやすい
- 名前を変えるときは、エディタの「名前の変更」機能で使用箇所をまとめて変える

注意

Inspector に出ている変数は、名前を変えると設定した値がリセットされる。変えた後に Inspector を確認する。

意味の分からない数字が直接書いてある

C# GameManager.cs

BEFORE

```
1 void Update()
2 {
3     if (score ≥ 1000)
4         ShowClear();
5     if (player.position.y < -8f)
6         Respawn();
7 }
```

C# GameManager.cs

AFTER

```
1 const int ClearScore = 1000;
2 [SerializeField] float fallLimitY = -8f;
3
4 void Update()
5 {
6     if (score ≥ ClearScore)
7         ShowClear();
8     if (player.position.y < fallLimitY)
9         Respawn();
10 }
```

- 数字に名前がつくと「クリアに必要なスコア」「落下判定の高さ」だと読める
- 調整したい値は **[SerializeField]** で Inspector から変えられるようにする

POINT

同じ数字が何か所にも書かれていると、1か所だけ変え忘れてバグになる。名前をつければ変更は1か所で済む。

同じ処理がコピペで何度も出てくる

C# Player.cs

BEFORE

```
1 void OnHitEnemy()  
2 {  
3     hp -= 10;  
4     hpText.text = "HP: " + hp;  
5     if (hp ≤ 0) GameOver();  
6 }  
7  
8 void OnHitSpike()  
9 {  
10    hp -= 30;  
11    hpText.text = "HP: " + hp;  
12    if (hp ≤ 0) GameOver();  
13 }
```

C# Player.cs

AFTER

```
1 void OnHitEnemy() { TakeDamage(10); }  
2 void OnHitSpike() { TakeDamage(30); }  
3  
4 void TakeDamage(int damage)  
5 {  
6     hp -= damage;  
7     hpText.text = "HP: " + hp;  
8     if (hp ≤ 0) GameOver();  
9 }
```

POINT

違うのは数字だけ。違う部分を引数にすれば1つの関数にまとめ、HP 表示の変更も1か所で済む。

Update に全部書いてある

C# Player.cs

BEFORE

```
1 void Update()
2 {
3     // 移動の処理が 20 行 ...
4     // ジャンプの処理が 15 行 ...
5     // 攻撃の処理が 30 行 ...
6     // スコアの処理が 10 行 ...
7 }
```

C# Player.cs

AFTER

```
1 void Update()
2 {
3     Move();
4     Jump();
5     Attack();
6     UpdateScore();
7 }
8
9 void Move() { /* 移動だけ */ }
10 void Jump() { /* ジャンプだけ */ }
```

- Update を読むだけで「毎フレーム何をしているか」が分かる
- ジャンプがおかしいときは、Jump の中だけを見ればよくなる
- 目安：**1つの関数は1つの仕事**。画面をスクロールしないと読めない関数は分ける

if の中に if が何段も重なっている

C# Gun.cs

BEFORE

```
1 void Shoot()
2 {
3     if (!isDead)
4     {
5         if (ammo > 0)
6         {
7             if (timer ≤ 0f)
8             {
9                 Fire();
10            }
11        }
12    }
13 }
```

C# Gun.cs

AFTER

```
1 void Shoot()
2 {
3     if (isDead) return;
4     if (ammo ≤ 0) return;
5     if (timer > 0f) return;
6
7     Fire();
8 }
```

POINT

「できない場合」を先に return で抜ける（早期リターン）。本当にやりたい処理が、いちばん浅い位置に残る。

PART 3

これから気をつけること

プログラムを書くとき、ゲームを制作するときの習慣

3

コードを書くときに気をつけること

1

名前で意味を伝える

`a・tmp・flg`ではなく `hp・isDead`。迷ったら長い名前でよい。

2

数字に名前をつける

直接書いた `1000` や `-8f` は、定数か `Inspector` の変数にする。

3

コピペしたら関数を疑う

同じ処理がもう一度出てきたら、1つにまとめるチャンス。

4

1つの関数に1つの仕事

`Update` には「何をするか」だけを並べ、中身は別の関数に書く。

5

ネストは浅くする

条件に合わないときは先に `return`。if が3段になったら見直す。

POINT

全部を一度に守らなくてよい。まずは「名前」から。1か月後の自分が読める名前になっているかを考える。